

UNIT 1 — FUNDAMENTALS OF PROGRAMMING

Complete Consolidated Notes — All Uploaded Topics

Combined directly from Unit-1-Part-1, Unit-1-Part-2 and Unit-1-Part-3.

Problem Solving Techniques

Outline

Skills Required for Software Engineers

Types of Problems

Computational Problems

Problem Solving Life Cycle

Logic

Algorithms

Flowcharts

Programming Languages

Programming Environments

Pseudocode

Learning Outcomes

Understand what problem solving means.

Identify different types of problems.

Understand computational problems.

Explain the problem-solving life cycle.

Differentiate between positive and negative logic.

Design algorithms and flowcharts.

Write simple pseudocode.

Understand programming environments.

What is Problem Solving?

Definition

Problem solving is the process of identifying a problem, analyzing it, and finding the most effective solution.

In Computer Science,

Problem solving means converting a real-world problem into a sequence of logical steps that a computer can execute.

Example

Problem:

Find the largest number among three numbers.

Number is positive or negative (Flowchart)

Number is positive or negative (Algorithm)

Step 1: Read no.
Step 2: If no. is greater than equal zero,
go to step 4.
Step 3: Print no. is a negative number,
go to step 5.

Step 4: Print no. is a positive number.
Step 5: Stop.

Why is Problem Solving Important?

Why is Problem Solving Important?

Problem solving is one of the most essential skills for every software engineer. Every software application is designed to solve a specific problem faced by users or organizations. A programmer's job is not just to write code but to analyze problems, design efficient solutions, and implement them correctly.

Importance of Problem Solving

Helps understand the real problem before writing code.

Reduces errors and improves software quality.

Saves development time and cost.

Produces efficient and optimized solutions.

Makes programs easier to maintain and update.

Enhances logical and analytical thinking.

Why is Problem Solving Important?

Real-World Examples

Skills Required for a Software Engineer

Skills Required for a Software Engineer

A successful software engineer requires a combination of technical knowledge, logical thinking, and interpersonal abilities. These skills help in designing, developing, testing, and maintaining software effectively.

Three Major Skill Categories

Technical Skills: Knowledge of programming languages, software tools, databases, operating systems, and modern technologies required to build software.

Problem Solving Skills: The ability to analyze problems, think logically, and develop efficient algorithms and solutions.

Soft Skills: Personal and professional skills that help software engineers communicate, collaborate, and work effectively in teams.

Skills Required for a Software Engineer

Example

A software engineer developing an Online Shopping Application needs:

Technical Skills → Java, SQL, Database, APIs

Problem Solving Skills → Design efficient search and payment algorithms

Soft Skills → Communicate with clients and work with teammates

Technical Skills

Technical skills are the knowledge and practical abilities required to design, develop, test, and maintain software applications. These skills are acquired through education, practice, and hands-on experience.

Major Technical Skills

Programming Languages (C, C++, Java, Python)

Data Structures and Algorithms (DSA)

Database Management Systems (MySQL, PostgreSQL, Oracle)

Operating Systems (Windows, Linux)

Computer Networks

Version Control (Git & GitHub)

Software Development Tools (Visual Studio Code, Eclipse)

Debugging and Testing

Web and Mobile Development Basics

Technical Skills

Why are Technical Skills Important?

Convert ideas into software.

Improve coding efficiency.

Reduce software bugs.

Build secure and scalable applications.

Adapt to new technologies.

Problem Solving Skills

Problem-solving skills are the abilities that help a software engineer identify a problem, analyze its causes, think logically, and develop the most efficient solution.

A programmer's primary responsibility is not only to write code but also to solve real-world problems using computational thinking.

Key Problem-Solving Skills

Analytical Thinking

Logical Reasoning

Critical Thinking

Algorithm Design

Decision Making

Debugging Skills

Creativity and Innovation

Attention to Detail

Problem Solving Skills

Example

Problem: A website takes 15 seconds to load.

A good software engineer will:

Analyze the problem.

Identify the bottleneck (database, network, or code).

Evaluate multiple solutions.

Select the most efficient solution.

Test the improvement.

Problem Solving Process

Steps in Problem Solving

Every software engineer follows a structured process to solve problems effectively.

Characteristics of a Good Problem Solver

A successful software engineer develops habits that improve the quality of solutions.

Essential Characteristics

Thinks logically before coding.

Breaks complex problems into smaller parts.

Considers multiple solution approaches.

Chooses the most efficient algorithm.

Learns from mistakes and debugging.

Continuously improves existing solutions.

Remains patient while solving difficult problems.

Soft Skills

Soft skills are personal, interpersonal, and communication skills that enable software engineers to work effectively with colleagues, clients, and stakeholders.

Unlike technical skills, soft skills focus on how you work with people rather than technology.

Important Soft Skills

Communication Skills

Teamwork

Leadership

Time Management

Adaptability

Problem Ownership

Professional Ethics

Emotional Intelligence

Important Soft Skills for Software Engineers

Why Soft Skills Matter in the Software Industry

Importance of Soft Skills

Technical knowledge alone is not enough to become a successful software engineer.

Employers also evaluate how candidates communicate, collaborate, and solve workplace challenges.

Benefits of Strong Soft Skills

Better teamwork and collaboration.

Effective communication with clients.

Improved leadership potential.

Faster conflict resolution.

Better project management.

Increased career growth opportunities.

Higher customer satisfaction.

Identify the Required Skill

Problem and Types of Problems

A problem is a situation where there is a difference between the current state and the desired (expected) state, requiring a solution to achieve the desired outcome.

In computer science, a problem is a task or challenge that can be solved using logical thinking, algorithms, and programming.

General Structure of a Problem

Every problem consists of four basic components:

Current State – The existing situation.

Desired State (Goal) – The expected outcome.

Obstacles or Constraints – Factors that make the goal difficult to achieve.

Solution – A sequence of steps to overcome the obstacles.

Problem and Types of Problems

Example: Student Examination

Types of Problems

Classification of Problems

Problems can be classified into three major categories:

1. Social Problems

Problems that affect society or communities.

2. Management Problems

Problems related to planning, organizing, controlling, and decision-making in organizations.

3. Computational Problems

Problems that can be solved using computers, algorithms, and programming.

Social Problems

A social problem is an issue that affects a large number of people and negatively impacts society. These problems often require collective efforts from governments, organizations, and individuals.

Characteristics

Affect communities or society.

May involve economic, environmental, or cultural factors.

Often require long-term solutions.

Technology can assist but cannot solve them alone.

Social Problems

Technology-Based Solutions

Management Problems

Management problems involve the efficient planning, organizing, coordinating, and controlling of resources within an organization.

These problems focus on improving productivity, reducing costs, and making informed decisions.

Characteristics

Occur in businesses and organizations.

Involve resource allocation.

Require decision-making.

Often solved using Management Information Systems (MIS).

Management Problems

Examples

Employee scheduling

Payroll processing

Inventory management

Customer relationship management

Budget planning

Supply chain management

Computational Problems

A computational problem is a problem that can be solved using algorithms, data structures, and computer programs.

The solution is represented as a sequence of logical instructions that a computer can execute.

Characteristics

Clearly defined inputs and outputs.

Solved using computational methods.

Can be automated.

Requires algorithm design.

Computational Problems

Examples

Finding the largest number.

Searching for a student's record.

Sorting employee salaries.

Calculating electricity bills.

Predicting weather.

Face recognition.

Route optimization in Google Maps.

Spam email detection.

Comparison of Problem Types

Computational Problem Types

Types of Computational Problems

Computational problems are classified based on the type of output required.

The five commonly encountered computational problems are:

Decision Problems

Searching Problems

Sorting Problems

Counting Problems

Optimization Problems

Decision Problems

A decision problem is a computational problem where the output is one of two possible answers, typically Yes/No, True/False, or Accept/Reject.

Characteristics

Binary outcome.

Uses logical conditions.

Commonly implemented using if-else statements.

Examples

Is the entered password correct?

Searching Problems

A searching problem involves locating a required item within a collection of data.

Objective

Find whether the required element exists and determine its position.

Examples

Search a student by roll number.

Sorting Problems

Sorting is the process of arranging data in a specified order.

Types

Ascending Order

Descending Order

Example

Before Sorting

56 12 89 24 35

After Sorting (Ascending)

12 24 35 56 89

Sorting Problems

Why Sorting is Important?

Makes searching faster.

Improves data organization.

Simplifies report generation.

Enhances database performance.

Common Sorting Algorithms

Bubble Sort

Selection Sort

Insertion Sort

Merge Sort

Quick Sort

Counting Problems

A counting problem involves determining the total number of occurrences or items that satisfy a specific condition.

Examples

Count the number of students who passed.

Example

Array

5 8 12 17 22 31

Even numbers:

8 12 22

Count = 3

Optimization Problems

An optimization problem aims to find the best possible solution among many feasible solutions while satisfying given constraints.

The objective may be to minimize or maximize a particular value.

Objectives

Minimize cost

Minimize time

Minimize distance

Maximize profit

Maximize efficiency

Optimization Problems

Problem-Solving Life Cycle

The Problem-Solving Life Cycle is a systematic approach used to solve problems efficiently. It ensures that solutions are logical, accurate, and easy to implement.

Problem-Solving Life Cycle

Problem-Solving Life Cycle

Example – Student Result System

Identify the need to calculate grades.

Analyze grading rules.

Design the grading logic.

Write the algorithm.

Create a flowchart.

Develop the program in C.

Test with different marks.

Deploy to the college portal.

Update grading rules when the university changes its policy.

Logic and Problem-Solving Tools Introduction to Logic

What is Logic?

Logic is a systematic way of thinking and reasoning that helps us reach a correct conclusion or make a correct decision.

In problem solving, logic helps us determine:

What needs to be done?

In what order should it be done?

What conditions need to be checked?

What should happen when a condition is true or false?

Logic and Problem-Solving Tools Introduction to Logic

Simple Example

Problem: Check whether a student has passed.

If marks ≥ 40

Student has Passed

Else

Student has Failed

Here, the condition marks ≥ 40 represents the logical decision.

Key Idea

Logic provides the reasoning behind a solution, while programming provides the implementation of that solution.

Importance of Logic in Problem Solving

Why is Logic Important?

Logic is the foundation of effective problem solving and programming.

1. Helps Understand the Problem

Logic helps identify:

Inputs

Outputs

Conditions

Relationships

Constraints

2. Helps Design Solutions

Before writing code, we need to determine the correct sequence of actions.

3. Helps Make Decisions

Programs frequently need to make decisions.

Importance of Logic in Problem Solving

4. Helps Reduce Errors

Logical thinking allows us to identify incorrect conditions and missing cases before implementation.

5. Helps Develop Efficient Algorithms

Good logic can lead to:

Fewer steps

Less processing

Better performance

Easier maintenance

Logic in Everyday Life

Logic is Not Limited to Programming

We use logic in our daily lives without realizing it.

Conclusion: The same logical thinking used in daily decision-making is used to design computer programs.

Logical Thinking in Programming

How Logic Becomes a Program

A programmer generally follows this path (refer the right image):

Example

Problem: Determine whether a person can vote.

Logic:

If age ≥ 18

Eligible

Else

Not Eligible

Positive Logic

What is Positive Logic?

Positive logic is a logical representation in which the active or ON state is represented by 1 (HIGH) and the inactive or OFF state is represented by 0 (LOW).

Representation

Positive Logic

Example

Suppose a switch controls a light:

Switch = 1 $\rightarrow$ Light ON

Switch = 0 $\rightarrow$ Light OFF

Software Example

Consider a variable:

isLoggedIn = 1

This can represent:

User is logged in $\rightarrow$ TRUE

Similarly:

isLoggedIn = 0

means:

User is not logged in $\rightarrow$ FALSE

In positive logic:

1 represents the active/true state, while 0 represents the inactive/false state.

Positive Logic: Examples

Example 1 – Security System

1 → Alarm Active

0 → Alarm Inactive

Example 2 – Login System

1 → Login Successful

0 → Login Failed

Example 3 – Machine Status

1 → Machine Running

0 → Machine Stopped

Example 4 – Sensor

1 → Object Detected

0 → Object Not Detected

Positive logic is intuitive because the value 1 directly represents the active or TRUE condition.

Negative Logic

What is Negative Logic?

Negative logic is a logical representation in which the active or ON state is represented by 0 (LOW) and the inactive or OFF state is represented by 1 (HIGH).

Representation

Negative Logic

Example

Suppose a control signal is active-low:

Signal = 0 → Device ON

Signal = 1 → Device OFF

Here, the system becomes active when the signal is LOW.

Why is it called Negative Logic?

Because the logical meaning is inverted compared with positive logic.

Positive Logic vs Negative Logic

Positive Logic vs Negative Logic

Simple Visualization

POSITIVE LOGIC

1 ■■■■■■ ON

0 ■■■■■■ OFF

NEGATIVE LOGIC

0 ■■■■■■ ON

1 ■■■■■■ OFF

Important Note

The concept of positive and negative logic is particularly important in digital electronics and hardware control signals. In ordinary high-level programming, Boolean true and false are normally treated independently of electrical HIGH/LOW conventions.

Problem-Solving Tools

Tools Used for Problem Solving

Before writing a program, programmers commonly represent the solution using structured problem-solving tools.

Two Important Tools

1. Algorithm

A step-by-step procedure for solving a problem.

2. Flowchart

A graphical representation of the steps of an algorithm.

What is an Algorithm?

Definition of Algorithm

An algorithm is a finite sequence of clear, well-defined steps used to solve a problem or perform a task.

Simple Definition

Algorithm = Step-by-step procedure to solve a problem.

Example

Problem: Calculate the area of a rectangle.

What is an Algorithm?

Example

Problem: Calculate the area of a rectangle.

Step 1: Start

Step 2: Read length L

Step 3: Read width W

Step 4: Calculate Area = $L \times W$

Step 5: Display Area

Step 6: Stop

Characteristics of a Good Algorithm

A good algorithm should have the following characteristics:

1. Input: An algorithm may accept zero or more inputs.
2. Output: It should produce at least one meaningful result.
3. Definiteness: Every step must be clear and unambiguous.
4. Finiteness: The algorithm must terminate after a finite number of steps.
5. Effectiveness: Each step should be simple enough to be executed practically.
6. Correctness: The algorithm should produce the correct result for valid inputs.

Advantages of Algorithms

Why Do We Write Algorithms?

1. Easy to Understand

The solution is represented as a sequence of steps.

2. Language Independent

An algorithm does not depend on C, Java, Python, etc.

3. Helps Before Coding

It allows programmers to design the solution before writing code.

4. Easier Debugging

Logical errors can be identified before implementation.

5. Easier Communication

Developers can discuss the solution without focusing on programming syntax.

6. Easy Conversion to Code

An algorithm can be converted into almost any suitable programming language.

Algorithm vs Pseudocode

An algorithm is a conceptual, step-by-step logic used to solve a problem, whereas pseudocode is a text-based representation of an algorithm. Think of an algorithm as the actual blueprint or recipe in your mind, while pseudocode is the written draft using simple words.

Algorithm vs Pseudocode

Algorithm Example: Even or Odd

```
Problem
Determine whether a given number is even or odd.
Algorithm
Step 1: Start
Step 2: Read N
Step 3: Calculate R = N % 2
Step 4: If R = 0, then go to Step 5; otherwise, go to Step 7
Step 5: Display "Even"
Step 6: Go to Step 8
Step 7: Display "Odd"
Step 8: Stop
```

What is a Flowchart?

A flowchart is a graphical representation of an algorithm using standardized symbols and arrows to show the sequence of operations.

Why Use Flowcharts?

Flowcharts help us:

Visualize the solution.

Understand the sequence of operations.

Identify decisions and conditions.

Communicate the solution easily.

Find logical errors.

Explain algorithms to others.

Problem

↓

Algorithm

↓

Flowchart

↓

Program

What is a Flowchart?

Flowchart Example: Add Two Numbers

Algorithm vs Flowchart

Common Mistakes in Logic and Flowcharts

Mistakes to Avoid

Logical Mistakes

Incorrect condition

Missing condition

Wrong sequence of steps

Infinite loop

Incorrect calculation

Ignoring special cases

Mistakes to Avoid

Flowchart Mistakes

Missing Start/Stop

Incorrect symbols

Missing arrows

Decision without Yes/No paths

Confusing process and decision symbols

Flowchart does not match the algorithm

From Problem to Program

Algorithm, Flowchart & Pseudocode

Problem: Calculate the Average of Three Numbers

1. Algorithm

Step 1: Start

Step 2: Read three numbers A, B, and C

Step 3: Calculate the sum: $SUM = A + B + C$

Step 4: Calculate the average: $AVG = SUM / 3$

Step 5: Display AVG

Step 6: Stop

Algorithm, Flowchart & Pseudocode

Problem: Calculate the Average of Three Numbers

2. Pseudocode

BEGIN

 READ A, B, C

$SUM \leftarrow A + B + C$

$AVG \leftarrow SUM / 3$

 DISPLAY AVG

END

Algorithm, Flowchart & Pseudocode

Problem: Calculate the Average of Three Numbers

3. Flowchart (On right side)

Note: Important Message

Students should understand that coding is not the first step.

First think → then design → then code.

Common Mistakes in Logic and Flowcharts

Example

Incorrect:

Check age

↓

Eligible

There is no condition or alternative path.

Example

Correct:

Age ≥ 18 ?

/ \

Yes No

↓↓

Eligible Not Eligible

Examples of Flowchart & Algorithms

1. Electricity Bill Calculator

Problem:

Write an algorithm and draw a flowchart to calculate the electricity bill based on the number of units consumed.

Example rule:

Up to 100 units → ■5 per unit

101–200 units → ■7 per unit

Above 200 units → ■10 per unit

Examples of Flowchart & Algorithms

Algorithm

Step 1: Start.

Step 2: Read the number of units consumed, U.

Step 3: Check if $U \leq 100$.

If Yes, calculate:

Bill = $U \times 5$

If No, go to Step 4.

Step 4: Check if $U \geq 101$ AND $U \leq 200$.

If Yes, calculate:

Bill = $U \times 7$

If No, go to Step 5.

Step 5: Calculate:

Bill = $U \times 10$

Step 6: Display the electricity bill, Bill.

Step 7: Stop.

Examples of Flowchart & Algorithms

2. ATM Withdrawal

Problem:

Write an algorithm and draw a flowchart for an ATM withdrawal system.

Conditions:

Read account balance and withdrawal amount.

If withdrawal amount is greater than balance → Display "Insufficient Balance".

Otherwise, deduct the amount and display the remaining balance.

Example:

Balance = ■10,000, Withdrawal = ■3,500

Output: Remaining Balance = ■6,500

Examples of Flowchart & Algorithms

Algorithm

Step 1: Start.

Step 2: Read the account balance, Balance.

Step 3: Read the withdrawal amount, Withdrawal.

Step 4: Check if Withdrawal > Balance.

If Yes, display "Insufficient Balance" and go to Step 7.

If No, go to Step 5.

Step 5: Calculate the remaining balance:

Remaining Balance = Balance – Withdrawal

Step 6: Display Remaining Balance.

Step 7: Stop.

Examples of Flowchart & Algorithms

3. Movie Ticket Price Calculator

Problem:

Write an algorithm and draw a flowchart to calculate the movie ticket price based on the person's age.

Conditions:

Age below 5 → Free

Age 5–17 → ■100

Age 18–59 → ■200

Age 60 or above → ■120

Example:

Age = 65

Output: Ticket Price = ■120

Examples of Flowchart & Algorithms

Algorithm

Step 1: Start.

Step 2: Read the person's age, Age.

Step 3: Check if Age < 5.

If Yes, set Ticket Price = ■0 (Free) and go to Step 7.

If No, go to Step 4.

Step 4: Check if Age ≥ 5 AND Age ≤ 17.

If Yes, set Ticket Price = ■100 and go to Step 7.

If No, go to Step 5.

Step 5: Check if Age ≥ 18 AND Age ≤ 59.

If Yes, set Ticket Price = ■200 and go to Step 7.

If No, go to Step 6.

Step 6: Set Ticket Price = ■120.

Step 7: Display Ticket Price.

Step 8: Stop.

Introduction to Computer-Based Problem Solving

What is Computer-Based Problem Solving?

Computer-based problem solving is the process of using a computer, programming language, and systematic problem-solving techniques to develop a solution to a problem.

The computer does not directly understand a real-world problem. The problem must first be converted into a form that can be processed by a computer.

Introduction to Computer-Based Problem Solving

Example

Real-world problem: Calculate the electricity bill.

The problem is converted into:

Input → Units consumed

Processing → Apply billing rules

Output → Total bill amount

Computer-based problem solving converts a real-world problem into computational steps that a computer can execute.

Why Do We Need Programming Languages?

Importance of Programming Languages

Computers understand instructions in machine language, which consists of binary values such as 0 and 1.

Humans, however, find machine language difficult to write and understand.

Programming languages provide a bridge between human thinking and computer execution.

Evolution of Programming Languages

Evolution of Programming Languages

Programming languages have evolved to make software development easier, faster, and more reliable.

1. Machine Language

Uses binary instructions.

10110000 01100001

Difficult for humans.

2. Assembly Language

Uses symbolic instructions.

MOV AX, 5

ADD AX, 10

Evolution of Programming Languages

3. High-Level Languages

Closer to human language.

sum = a + b;

Examples: C, C++, Java and Python

4. Modern Languages

Designed for specific domains and modern software development.

Examples: Python → AI/Data Science, JavaScript → Web, Swift → iOS, Kotlin → Android, R → Statistical Computing

Importance of Studying Programming Languages

Why Should We Study Programming Languages?

Learning programming languages is important because it develops both programming ability and computational thinking.

1. Converts Logic into Programs

How to convert algorithms into executable code.

2. Develops Computational Thinking

Programming encourages to:

Break problems into smaller parts.

Identify patterns.

Develop algorithms.

Think logically.

Importance of Studying Programming Languages

3. Provides Knowledge of Different Programming Paradigms

Students can understand approaches such as:

Procedural Programming

Object-Oriented Programming

Event-Driven Programming

4. Helps Choose the Right Tool

Different problems require different languages.

5. Improves Career Opportunities

Programming knowledge is essential in areas such as:

Software Development

Data Science

Artificial Intelligence

Cybersecurity

Web Development

Mobile Development

Programming Language Selection

Does One Programming Language Solve Every Problem?

-> No.

Different programming languages have different strengths.

Role and Impact of Programming Languages

Programming languages influence how software is: Designed, Developed, Tested , Maintained and Optimized

Major Roles

1. Problem Implementation

Converts algorithms into executable programs.

2. Software Development

Used to create applications, websites, operating systems, and other software.

3. Automation

Allows repetitive tasks to be performed automatically.

4. Data Processing

Enables computers to process and analyze large amounts of data.

5. Communication with Hardware

Languages such as C and C++ are widely used for systems and embedded programming.

Impact of Programming Languages

The choice of programming language can affect:

1. Performance

Some languages are better suited for high-performance applications.

2. Development Speed

High-level languages can reduce the amount of code required.

3. Security

Language features can influence how easily certain programming errors occur.

4. Maintainability

Readable and well-structured languages can make software easier to maintain.

5. Portability

Some languages allow programs to run across different operating systems and platforms.

6. Ecosystem

Libraries, frameworks, tools, and community support influence development productivity.

Programming Environment

A programming environment is a collection of software tools and facilities that help programmers write, compile, execute, test, debug, and maintain programs.

1. Compiler: A compiler translates the entire program at once into a standalone file before running, making execution faster.
2. Interpreter: An interpreter translates and runs code line by line during runtime, making debugging easier

Programming Environment

1. How a Compiler Works

Full Translation: Reads and converts the entire source code into machine code all at once before execution.

Creates Files: Generates a separate, standalone executable file (like an .exe file).

Execution Speed: Runs much faster because the translation is already finished.

Error Reporting: Displays all syntax errors together after scanning the entire code.

Examples: C, C++, and Go.

2. How an Interpreter Works

Line-by-Line: Translates and executes the code one line at a time as the program runs.

No File Created: Does not generate a separate permanent machine code file.

Execution Speed: Runs slower because translation happens simultaneously during runtime.

Error Reporting: Stops or reports errors instantly as it encounters them on a specific line.

Examples: Python, JavaScript, and Ruby.

Integrated Development Environment (IDE)

An Integrated Development Environment (IDE) is a software application that combines several programming tools into a single environment.

Typical IDE Features

Code Editor

Syntax Highlighting

Auto-completion

Compiler/Interpreter Integration

Debugger

Project Management

Error Detection

Build Tools

Version Control Integration

Without IDE:

Editor → Compiler → Terminal → Debugger

Program Design

Program design is the process of planning the structure, logic, data, and steps of a program before implementation.

The goal is to create a solution that is:

Correct

Efficient

Understandable

Maintainable

Testable

Can anyone draw a Program Design Process?

Program Design Example

Problem

Develop a program that accepts marks and determines the student's grade.

Step 1 – Input

Marks

Step 2 – Processing

Determine grade based on marks.

Example:

90–100 → A

80–89 → B

70–79 → C

60–69 → D

< 60 → F

Step 3 – Output

Grade

Step 4 – Logic

Read marks

↓

Check range

↓

Determine grade

↓

Display grade

Step 5

Convert the logic into:

Algorithm → Flowchart → Pseudocode → Program

Program Design and Implementation Issues

Before implementing a program, programmers need to consider several issues.

1. Correctness: Does the program produce the expected result?
2. Efficiency: Does it use reasonable: 1) Time? 2)Memory? 3)CPU resources?
3. Scalability: Can the program handle increasing amounts of data or users?
4. Maintainability: Can developers easily modify and update the program?
5. Reliability: Does the program behave consistently?
6. Usability: Can users understand and operate the system easily?
7. Security: Does the program protect data and prevent unauthorized access?

Implementation Issues

Even a well-designed solution can face problems during implementation.

Common Issues

Syntax errors

Logical errors

Runtime errors

Incorrect input handling

Memory problems
Performance problems
Poor error handling
Security vulnerabilities
Compatibility issues

Implementation Issues

```
Types of Errors
Syntax Error
printf("Hello"
Missing ");"
Logical Error
average = total * count;
The intended operation should be division.
```

Runtime Error

Example:

Division by zero

Key Point

Testing and debugging are essential parts of implementation.

What is Pseudocode?

Pseudocode is an informal, structured representation of an algorithm written using simple English-like statements and programming concepts.

It describes what the program should do without following the exact syntax of a particular programming language.

Example

Problem:

Find the larger of two numbers.

What is Pseudocode?

```
START
READ A
READ B
IF A > B THEN
    DISPLAY A
ELSE
    DISPLAY B
END IF
STOP
```

Important Characteristics

Easy to understand.

Language independent.

Focuses on logic rather than syntax.

Easy to convert into actual program code.

Pseudocode Keywords

There is no single universal syntax for pseudocode. Different textbooks and organizations may use slightly different conventions.

```
START
    READ number
    IF number > 0 THEN
        DISPLAY "Positive"
    ELSE
        DISPLAY "Not Positive"
    END IF
STOP
```

Pseudocode Example: Find Largest of Three Numbers

```
START
READ A, B, C
IF A >= B AND A >= C THEN
    largest = A
```

```
ELSE IF B >= A AND B >= C THEN
    largest = B
ELSE
    largest = C
END IF
DISPLAY largest
STOP
```

Pseudocode with Repetition

Example: Calculate Sum of First N Numbers

Problem

Calculate:

$1 + 2 + 3 + \dots + N$

```
START
READ N
SET sum = 0
SET i = 1
WHILE i <= N
    sum = sum + i
    i = i + 1
END WHILE
DISPLAY sum
STOP
```

Algorithm vs Pseudocode vs Flowchart

Advantages and Limitations of Pseudocode

Advantages

Easy to read.

Easy to modify.

Language independent.

Focuses on problem logic.

Helps detect logical errors.

Makes communication easier.

Easy to convert into code.

Limitations

No universally standardized syntax.

Different people may write it differently.

Cannot be directly executed by a computer.

May become ambiguous if poorly written.

Complex systems may require additional design documentation.

UNIT 1 — EXAM REVISION CHECKLIST

- Problem solving: definition, importance, real-world examples and characteristics of a good problem solver
- Skills required for software engineers: technical, problem-solving and soft skills
- Problems: current state, desired state, constraints and solution
- Social, management and computational problems; comparison and examples
- Computational problem types: decision, searching, sorting, counting and optimization
- Problem-solving life cycle: identification → analysis → design → algorithm → flowchart/pseudocode → coding → testing/debugging → deployment → maintenance
- Logic: meaning, importance, everyday logic and logic in programming
- Positive and negative logic, active/inactive states and examples
- Algorithms: definition, characteristics, advantages and examples
- Flowcharts: definition, uses, symbols, examples and common mistakes
- Algorithm vs pseudocode vs flowchart
- Pseudocode: definition, characteristics, keywords, examples, repetition, advantages and limitations
- Computer-based problem solving: input → processing → output
- Programming languages: machine, assembly, high-level and modern/domain-specific languages
- Programming language selection, roles, impact and applications
- Programming environments: compiler, interpreter and IDE
- Program design: input, processing, output and logic
- Implementation issues: correctness, efficiency, scalability, maintainability, reliability, usability and security
- Errors: syntax, logical and runtime errors